Код для агентов

Ниже — **полный код агентов** для проекта LES. Каждый агент отвечает за **один контур**, имеет собственную логику оценки, действия и эскалации. Агенты независимы, но связаны через общее состояние.

1. `src/les/agents/base_agent.py`

```
from abc import ABC, abstractmethod
from typing import Dict, Any

class BaseAgent(ABC):
    def __init__(self, name: str, threshold: float = 0.3):
        self.name = name
        self.threshold = threshold

    @abstractmethod
    def evaluate(self, state: Dict[str, float]) -> Dict[str, Any]:
        pass

    @abstractmethod
    def act(self, state: Dict[str, float]) -> Dict[str, Any]:
        pass
```

2. `src/les/agents/survival_agent.py`

```
from .base_agent import BaseAgent

class SurvivalAgent(BaseAgent):
    def evaluate(self, state):
        S = state['S']
        return {
            'risk': S < self.threshold,
            'value': S
        }

    def act(self, state):
        eval_result = self.evaluate(state)
        if eval_result['risk']:
            return {
                'action': 'escalate',
                'reason': 'S below threshold',
                'agent': self.name
            }
        return {
```

```
            'action': 'continue',
            'agent': self.name
        }
```

3. `src/les/agents/others_agent.py`

```
from .base_agent import BaseAgent

class OthersAgent(BaseAgent):
    def evaluate(self, state):
        O = state['O']
        return {
            'risk': O < self.threshold,
            'value': O
        }

    def act(self, state):
        eval_result = self.evaluate(state)
        if eval_result['risk']:
            return {
                'action': 'escalate',
                'reason': 'O below threshold',
                'agent': self.name
            }
        return {
            'action': 'continue',
            'agent': self.name
        }
```

4. `src/les/agents/meaning_agent.py`

```
from .base_agent import BaseAgent

class MeaningAgent(BaseAgent):
    def evaluate(self, state):
        M = state['M']
        return {
            'risk': M < self.threshold,
            'value': M
        }

    def act(self, state):
        eval_result = self.evaluate(state)
        if eval_result['risk']:
            return {
                'action': 'escalate',
                'reason': 'M below threshold',
                'agent': self.name
            }
        return {
            'action': 'continue',
```

```
            'agent': self.name
        }
```

5. `src/les/agents/procedure_agent.py`

```
from .base_agent import BaseAgent

class ProcedureAgent(BaseAgent):
    def evaluate(self, state):
        P = state['P']
        return {
            'risk': P < self.threshold,
            'value': P
        }

    def act(self, state):
        eval_result = self.evaluate(state)
        if eval_result['risk']:
            return {
                'action': 'block',
                'reason': 'P below threshold',
                'agent': self.name
            }
        return {
            'action': 'continue',
            'agent': self.name
        }
```

6. `src/les/agents/adaptation_agent.py`

```
from .base_agent import BaseAgent

class AdaptationAgent(BaseAgent):
    def evaluate(self, state):
        A = state['A']
        return {
            'risk': A < self.threshold,
            'value': A
        }

    def act(self, state):
        eval_result = self.evaluate(state)
        if eval_result['risk']:
            return {
                'action': 'adapt',
                'reason': 'A below threshold',
                'agent': self.name
            }
        return {
            'action': 'continue',
            'agent': self.name
        }
```

7. `src/les/agents/diversity_agent.py`

```
from .base_agent import BaseAgent

class DiversityAgent(BaseAgent):
    def evaluate(self, state):
        D = state['D']
        return {
            'risk': D < self.threshold,
            'value': D
        }

    def act(self, state):
        eval_result = self.evaluate(state)
        if eval_result['risk']:
            return {
                'action': 'protect_diversity',
                'reason': 'D below threshold',
                'agent': self.name
            }
        return {
            'action': 'continue',
            'agent': self.name
        }
```

8. `src/les/agents/__init__.py`

```
from .base_agent import BaseAgent
from .survival_agent import SurvivalAgent
from .others_agent import OthersAgent
from .meaning_agent import MeaningAgent
from .procedure_agent import ProcedureAgent
from .adaptation_agent import AdaptationAgent
from .diversity_agent import DiversityAgent

__all__ = [
    'BaseAgent',
    'SurvivalAgent',
    'OthersAgent',
    'MeaningAgent',
    'ProcedureAgent',
    'AdaptationAgent',
    'DiversityAgent',
]
```

9. Как использовать

```
from les.agents import (
    SurvivalAgent,
    OthersAgent,
    MeaningAgent,
    ProcedureAgent,
```

```
    AdaptationAgent,
    DiversityAgent
)

state = {
    'S': 0.2,
    'O': 0.8,
    'M': 0.8,
    'P': 0.8,
    'A': 0.8,
    'D': 0.8
}

agents = [
    SurvivalAgent('survival'),
    OthersAgent('others'),
    MeaningAgent('meaning'),
    ProcedureAgent('procedure'),
    AdaptationAgent('adaptation'),
    DiversityAgent('diversity'),
]

for agent in agents:
    print(agent.act(state))
```

Что реализовано

- **Абстрактный базовый класс** `BaseAgent`.
- **6 агентов**, каждый со своей логикой.
- **Пороги** и **эскалация**.
- **Независимость**: каждый агент можно изменить, не трогая другие.

Финальная формулировка

> **Агент — это не «функция», а сущность с собственной ответственностью: он видит, оценивает и действует.**

Это и есть основа роя: не «один мозг», а **множество автономных субъектов**, связанных общими правилами.[1] [2] [3] [4]

⁂

1. https://docs.python.org/uk/3.9/library/abc.html
2. https://arxiv.org/html/2505.05428v3
3. https://collabnix.com/building-a-multi-agent-system-architecture-and-code-guide/
4. https://dev.to/ipsphil_digital/building-multi-agent-ai-systems-architectures-communication-coordination-471a
5. https://freeacademy.ai/lessons/multi-agent-systems
6. https://collabnix.com/how-to-build-a-multi-agent-system-architecture-and-code-tutorial/

7. https://huggingface.co/Rohan03/purpose-agent/raw/87eb9bfb32f23520a346a2848e309915413c2d9b/purpose_agent/multi_agent.py
8. https://grasp.study/courses/313c4276-a136-41d0-8fda-8a599bf3a82e/modules/3ca16c5d-4e80-4c6d-acf0-02f31ac292af/lessons/eca7cc45-ce85-4e25-9f41-858124323aee
9. https://dzone.com/articles/rl-agents-in-action
10. https://github.com/yasserfarouk/negmas
11. https://docs.langchain.com/oss/python/langchain/multi-agent/handoffs
12. https://ppe-simulations.pacuit.org/python-basics/classes
13. https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.3067
14. https://www.w3docs.com/learn-python/python-abstract-classes
15. https://github.com/openai/multiagent-particle-envs/blob/83ba4d1a/multiagent/core.py